16.Comparison of Different Classification Algorithms

Aim

To implement and compare different **classification algorithms** in Python and evaluate their performance using **accuracy**.

Algorithm

1. Load the Iris dataset.
2. Divide the dataset into training and testing data.
3. Apply different classification algorithms:
    - K-Nearest Neighbours (KNN)
    - Decision Tree
    - Naïve Bayes
    - Support Vector Machine (SVM)
4. Train each algorithm using the training data.
5. Predict the classes using the testing data.
6. Calculate the accuracy of each algorithm.
7. Compare the performance of all algorithms.

PROGRAM:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


iris = load_iris()


X = iris.data

y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = {
    "KNN": KNeighborsClassifier(n_neighbors=3),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    print(name)
    print("Accuracy:", accuracy * 100, "%")
    print()
```

OUTPUT:

```
    "SVM": SVC()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    print(name)
    print("Accuracy:", accuracy * 100, "%")
    print()
```

```
KNN
Accuracy: 100.0 %

Decision Tree
Accuracy: 100.0 %

Naive Bayes
Accuracy: 100.0 %

SVM
Accuracy: 100.0 %
```

Result Statement

Thus, different **classification algorithms** were successfully implemented and compared using the Iris dataset. **KNN, Decision Tree, Naïve Bayes, and SVM** achieved high classification performance on the test dataset.